

Initial Working Report: Discrete Logarithm Algorithms

Owner: Sharique & Team **Date:** April 17, 2026

1. Project Progress and Empirical Results

We have successfully modeled, implemented, and tested four distinct algorithmic paradigms to solve the Discrete Logarithm Problem ($g^x \equiv h \pmod p$), which forms the bedrock of Diffie-Hellman Key Exchange security.

- 1. **Naive Brute Force Search** ($O(P)$ time, $O(1)$ memory)
- 2. **Baby-step Giant-step** ($O(\sqrt{P})$ time, $O(\sqrt{P})$ memory)
- 3. **Pollard's Rho Algorithm** ($O(\sqrt{P})$ time, practically $O(1)$ memory)
- 4. **Pohlig-Hellman Algorithm** (Time heavily dependent on the largest prime factor of $P-1$)

Benchmark Results

The initial Python testing suite (`dlp_algorithms.py`) confirmed our algebraic correctness.

Modulus Size	Modulus Value (P)	Brute Force	BSGS	Pollard's Rho	Pohlig-Hellman
16-bit	65,537	~0.0017s	0.0002s	0.0044s	0.0001s
20-bit	1,048,583	~0.0605s	0.0009s	0.0009s	0.0001s
24-bit	16,777,259	~2.2773s	0.0043s	0.0083s	0.0024s
26-bit (Smooth)	106,696,591	~2.3259s	0.0140s	0.0060s	0.0001s

Note: The 26-bit prime was engineered to be highly "smooth". Its group order mathematically factors cleanly into very small sub-primes: $P-1 = 2 \times 3 \times 5 \times 7 \times 11^2 \times 13 \times 17 \times 19$.

Significance of Current Findings

Our initial results are a textbook demonstration of cryptographic vulnerability. Despite the exponential search space expansion at 26-bits, **Pohlig-Hellman executed in fractions of a millisecond (\$0.000122s\$)**—over 100 times faster than the highly-optimized Baby-step Giant-step approach. By converting the problem utilizing the Chinese Remainder Theorem, we've empirically proven why cybersecurity protocols mandate the use of "Safe Primes" to prevent bad-actors from cracking encryption keys trivially.

2. Future Work & Extensive Testing Improvements

To elevate the project into a comprehensive, final-grade submission for CSE 317, we will expand our testing methodology to prove theoretical limits on consumer hardware.

A. Deep-Scale Dataset Stress Testing

- **Pushing Memory Constraints:** Generate 32-bit, 40-bit, and 48-bit "Safe Primes". The primary objective is to crash or stall the Baby-step Giant-step algorithm by overflowing system RAM boundaries via its

$O(\sqrt{P})$ hash map architecture.

- **Pollard's Rho Validation:** We will demonstrate that Pollard's Rho gracefully handles the massive scale where BSGS crashes, solely due to its $O(1)$ space complexity.
- **Timeout Benchmarking:** Determine the exact bit-length threshold where the Brute Force array crosses a "1-hour timeout," cementing its lack of scalability.

B. Analytical Telemetry and Visualization

- **Automated CSV Data Export:** Upgrade the Python test suite to programmatically output arrays of `[Bit_Length, Algorithm_Name, Execution_Time, Result_Status]` directly into `.csv` files.
- **Graphing (Matplotlib):** Create line and scatter plots specifically mapping Empirical Runtime (Y-axis) against the theoretical mathematical bounds (X-axis) to validate asymptotic complexity analysis concepts learned in class.

C. Advanced Implementation Improvements

- **Message Passing / Parallel Processing:** Leverage concepts from PDC (Parallel Distributed Computing) classrooms by parallelizing the Brute Force algorithm. We can use Python's Multiprocessing or **MPI** wrappers to partition the mathematical search space $[1...P]$ across multiple CPU cores, theoretically dividing the brute force time by N threads.
- **Enhanced Pohlig-Hellman:** Currently, the inner sub-step solvers utilize linear approximation. We can refactor Pohlig-Hellman to nest Baby-step Giant-step internally for its subgroups, making it robust against primes that are only semi-smooth.